Sprint 16 : React Learning Dashboard: Capstone Integration.

මෙම අධ්‍යයන මාර්ගෝපදේශය (Study Guide) සකස් කර ඇත්තේ Sprint 16 Capstone Project එකට අදාළ සියලුම සිද්ධාන්ත, කේත උදාහරණ (Code Examples) සහ තාක්ෂණික සංකල්ප (Technical Concepts) ඉතා සරලව සහ පැහැදිලිව දේශීය තාක්ෂණික උපකාරකයෙකුගේ (Technical Tutor) ඇසින් වටහා දීම සඳහාය.

S0. Where You Are Before Sprint 16 (Sprint 16 එකට කලින් ඔයා ඉන්න තැන)

- **Concept Explanation (Sinhala):** Sprint 16 එකට එන්න කලින්, ඔයාට React වල තියෙන මූලිකම සහ වැදගත්ම දේවල් (Sprints 1 ඉඳන් 15 වෙනකන්) ගැන හොඳ අවබෝධයක් තියෙන්න ඕනේ. ඔයා මේ වෙනකොට ඉගෙනගෙන තියෙන දේවල් තමයි:
  - React orientation, SPAs (Single Page Applications), සහ Vite setup කරන හැටි.
  - JSX, Components, Fragments, imports, සහ exports.
  - Props, children, composition, conditions, lists, keys, සහ inline styles.
  - JSX වලදී පාවිච්චි කරන accessibility habits.
  - Events සහ form inputs handle කරන හැටි.
  - useState, rendering, immutable state updates, සහ useReducer වල basic දේවල්.
  - Refs, effects, cleanup functions, custom hooks, සහ Context API.
  - Data fetching, backend integration basics, loading/error/data states, action state, සහ optimistic UI.
  - useMemo, useCallback, සහ React.memo පාවිච්චි කරලා කරන memoization (performance optimization).
  - npm සහ package.json හරහා dependency management කරන හැටි.
  - React Router පාවිච්චි කරලා normal routes, dynamic routes, nested routes, protected routes, 404 routes, සහ lazy-loaded routes හදන හැටි.
  - Shared state, prop drilling, Context, Redux concepts, Zustand concepts, සහ React DevTools.
  - CSS frameworks, preprocessors, සහ styling strategy.
  - Performance foundations, browser rendering, INP (Interaction to Next Paint), සහ sustainability.
  - Performance optimization, metrics, tools, Web APIs, image optimization, caching, compression, සහ CSS/JavaScript optimization.
  - Testing foundations, unit tests, React Testing Library, TDD/BDD, assertions, mocks, stubs, සහ fakes.
  - Testing strategy: functional, E2E, usability, compatibility, performance, security, A/B, alpha/beta, regression, සහ retesting.

  කලින් sprints වලදී ඔයා මේ හැම skill එකක්ම වෙන වෙනම තමයි ඉගෙනගත්තේ. හැබැයි Sprint 16 වලදී ඔයාට කරන්න තියෙන්නේ මේ හැම skill එකක්ම එකතු කරලා එක පොඩි, හැබැයි complete React app එකක් හදන්නයි. මේක තමයි ඔයාගේ **capstone sprint** එක.

S1. Sprint 16 Goal (Sprint 16 වල ඉලක්කය)

- **Concept Explanation (Sinhala):** මේ sprint එකේ ප්‍රධානම goal එක තමයි React Learning Dashboard කියලා කුඩා capstone project එකක් හදන එක. මේ app එකෙන් user කෙනෙක්ට පහත දේවල් කරන්න පුළුවන් වෙන්න ඕනේ:
  - ඉගෙනගන්න තියෙන React topics බලාගන්න (View a list of topics).
  - එක topic එකක් select කරලා ඒකේ detail page එකක් open කරගන්න (Open one topic detail page).
  - ඉගෙනගෙන ඉවර කරපු topics "completed" කියලා mark කරන්න.
  - තමන්ගේ මුළු learning progress එක dashboard එකෙන් බලාගන්න.
  - Practice කරන්න ඕන දේවල් ලියාගන්න plan එකක් හදාගන්න (Write a practice plan).
  - Routes පාවිච්චි කරලා පිටු අතර ලේසියෙන් navigate වෙන්න.
  - අවශ්‍ය තැන්වලදී shared state සහ local state හරියට පාවිච්චි කරන්න.
  - අඩුම ගානේ එක custom hook එකක් හෝ loading/error/data pattern එකක් පාවිච්චි කරන්න.
  - Props සඳහා පොඩි TypeScript example එකක් පාවිච්චි කරන්න.
  - Unit test එකක් සහ සරල testing strategy එකක් ඇතුළත් කරන්න.
  - Accessibility සහ performance checks ඇතුළත් කරන්න.

  සරලවම කිව්වොත්, කලින් ඉගෙනගත්ත හැම React පාඩමක්ම එකතු කරලා ඇත්තටම වැඩ කරන app එකක් හදන එක තමයි මෙතනදී කරන්නේ.

S2. The Big Sprint 16 Idea (Integration කියන ලොකු concept එක)

- **Concept Explanation (Sinhala):** මෙහි ප්‍රධානම අදහස තමයි "Integration" (එකතු කිරීම). කලින් ඉගෙනගත්ත routing, state, forms, Context, testing, සහ performance කියන concept ඔක්කොම එකතු වෙලා වැඩ කරන්නේ මෙහෙමයි:
  - Routes වලින් පිටු (pages) පෙන්වනවා.
  - State එකෙන් topic progress එක store කරගන්නවා.
  - Context එකෙන් ඒ topic progress එක app එක පුරා share කරනවා.
  - Forms වලින් practice plans ටික collect කරගන්නවා.
  - Utility function එකකින් completed topics ප්‍රමාණය calculate කරනවා.
  - Test එකකින් ඒ utility function එක හරියට වැඩ කරනවද කියලා check කරනවා.
  - Accessibility වලින් app එක ඕනෑම කෙනෙක්ට පාවිච්චි කරන්න පුළුවන් තත්ත්වයට පත් කරනවා.
  - Performance checks වලින් app එක වේගවත් කරනවා.
  - TypeScript props මගින් අපේ props වලට ආරක්ෂාවක් ලබා දෙනවා.

  ඇත්තම React development වලදී අපිට කවදාවත් එක concept එකක් විතරක් තනිවම හම්බවෙන්නේ නැහැ. මේ වගේ concept කිහිපයක් එක පාර පාවිච්චි කරන්න වෙනවා.

S3. Important Correction: Small but Complete (ලොකු app එකක්ද පොඩි app එකක්ද?)

- **Concept Explanation (Sinhala):** ආධුනිකයන් (beginners) කරන ලොකුම වැරැද්දක් තමයි "Capstone" කිව්ව ගමන් සෑහෙන්න ලොකු, complex application එකක් හදන්න යන එක. ඒක වැරදියි.

  වැදගත්ම දේ තමයි: **ලොකු භාගෙට වැඩ කරන app එකකට වඩා, පොඩි හරියටම වැඩ කරන complete app එකක් වටිනවා.**

  හොඳ Sprint 16 app එකක ලක්ෂණ:
  - පිටු (pages) සීමිත ප්‍රමාණයක් තිබීම.
  - පිරිසිදු දත්ත (Clear data).
  - හරියට වැඩ කරන routes.
  - නිවැරදි state updates.
  - වැඩ කරන forms.
  - එක ප්‍රයෝජනවත් unit test එකක්.
  - පිරිසිදු ව්‍යුහය (Clear structure).
  - සරල accessibility සහ performance checks.

  ආධුනිකයින්ගේ රීතිය (Beginner rule): App එක තවදුරටත් ලොකු කරන්න (expand) කලින්, දැනට තියෙන කොටස සම්පූර්ණයෙන්ම ඉවර කරන්න (Finish before expanding).

S4. Project Name: React Learning Dashboard (ප්‍රොජෙක්ට් එකේ අදහස)

- **Concept Explanation (Sinhala):** අපේ project එක තමයි **React Learning Dashboard**. මේක React ඉගෙනගන්න කෙනෙක්ට තමන්ගේ ප්‍රගතිය (progress) බලාගන්න පුළුවන් tracker එකක්. මේ app එකෙන් මේ ප්‍රශ්න වලට උත්තර ලැබෙනවා:
  - මම දැනට ඉගෙනගන්න React topics මොනවාද?
  - මම දැනටමත් සම්පූර්ණ කරලා තියෙන්නේ මොනවාද?
  - තවදුරටත් පුහුණු විය යුතු (need practice) topics මොනවාද?
  - මගේ මුළු progress එක කොහොමද?
  - අද මම practice කරන්න ඕනේ මොනවාද?

  පිටු සැකැස්ම (Pages breakdown):
  - **Dashboard page:** Progress එක සහ completed topics ගණන පෙන්වයි.
  - **Topics page:** සියලුම React topics සහ ඒවායේ done/not done status පෙන්වයි.
  - **Topic detail page:** URL එක අනුව අදාළ topic එකේ විස්තර පෙන්වන අතර, ඒක complete කියලා mark කරන්න ඉඩ දෙයි.
  - **Practice planner page:** දවසේ කරන්න ඕන පුහුණුවීම් ලියා තැබීමට form එකක් ලබා දෙයි.

S5. Sprint 16 Definition of Done (වැඩේ ඉවරයි කියන්නේ කවදාද?)

- **Concept Explanation (Sinhala):** ඔයාගේ project එක සම්පූර්ණයි (done) කියන්න නම් මේ දේවල් ටික අනිවාර්යයෙන්ම තියෙන්න ඕනේ:
  - App එක කිසිම crash එකක් නැතුව run වෙන්න ඕනේ.
  - Routes හරියට වැඩ කරන්න ඕනේ.
  - Topics ටික data වලින් render වෙන්න ඕනේ.
  - Topic detail page එක dynamic route එකක් වෙන්න ඕනේ (`/topics/:topicId`).
  - Topic එකක් completed කියලා mark කරන්න පුළුවන් වෙන්න ඕනේ.
  - State updates සියල්ලම immutable (පරණ state එක කෙලින්ම modify නොකරන) ඒවා වෙන්න ඕනේ.
  - Shared state එක සඳහා Context API පාවිච්චි වෙන්න ඕනේ.
  - Form data සඳහා local state පාවිච්චි වෙන්න ඕනේ.
  - Form එකක් validation සමඟ වැඩ කරන්න ඕනේ.
  - එක custom hook එකක් හෝ loading/error/data pattern එකක් තියෙන්න ඕනේ.
  - අඩුම තරමේ එක utility function එකක් සහ ඒකට unit test එකක් තියෙන්න ඕනේ.
  - TypeScript props පාවිච්චි කරපු components එකක් හෝ දෙකක් තියෙන්න ඕනේ.
  - Accessibility සහ performance checks කරලා තියෙන්න ඕනේ.

S6. Scrum Rhythm for Sprint 16 (වැඩ කරන රිද්මය)

- **Concept Explanation (Sinhala):** ඔයා මේ පිළිවෙළට වැඩ කරන්න පුරුදු වෙන්න: `Plan -> Learn -> Build -> Debug -> Review -> Refactor -> Retrospective`
  - **Plan:** App එකෙන් වෙන්න ඕන දේවල් කලින්ම තීරණය කරන්න. තිකන් හිතුණ පලියට features එකතු කරන්න එපා.
  - **Learn:** අවශ්‍ය React concepts (routing, context, states, hooks, etc.) කලින් නැවත මතක් කරගන්න.
  - **Build:** සරලම වැඩ කරන version එක (simplest working version) මුලින්ම හදන්න.
  - **Debug:** වැරදුණු routes, imports, state mutations, missing keys වගේ ප්‍රශ්න විසඳන්න.
  - **Review:** හැම requirement එකක්ම cover වෙලාද බලන්න.
  - **Refactor:** කේතය පිරිසිදු කරන්න (Clean up names, structure, and logic).
  - **Retrospective:** තමන්ගෙන්ම අහන්න: මට තේරුණු දේ මොකක්ද? අමාරු වුණේ කොතනද? ඊළඟ පාර දියුණු කරන්නේ කොහොමද? කියලා.

S7. Beginner App Architecture (සරල React App Architecture එක)

- **Concept Explanation (Sinhala):** කේතය (code) සහ components එකිනෙක සම්බන්ධ වෙලා තියෙන ආකාරය (Architecture) සරලව පහත පරිදි වේ:

```
[ App (Root) ]
└── [ BrowserRouter (Routing) ]
    └── [ LearningProvider (State) ]
        ├── [ / ] ──→ [ Dashboard ]
        ├── [ /topics ] ──→ [ TopicList ]
        ├── [ /topics/:topicId ] ──→ [ TopicDetail ]
        ├── [ /practice ] ──→ [ PracticePlanner ]
        └── [ * ] ──→ [ NotFound ]
```

S8. Suggested Folder Structure (නිවැරදි ෆෝල්ඩර් ව්‍යුහය)

- **Concept Explanation (Sinhala):** ප්‍රොජෙක්ට් එක පිළිවෙළකට තියාගන්න මේ folder structure එක පාවිච්චි කරන්න:
  - `components/` : නැවත පාවිච්චි කළ හැකි (reusable) UI කොටස් (උදා. Navbar, Layout, Button, etc.)
  - `pages/` : route-level screens හෙවත් සම්පූර්ණ පිටු (Dashboard, TopicList, etc.)
  - `context/` : app එක පුරා share කරන state (LearningContext)
  - `hooks/` : reusable hook logic (useFetch)
  - `utils/` : pure helper functions (countDone)
  - `data/` : static sample data (topics)

S9. Topic Data (දත්ත සැකැස්ම)

- **Concept Explanation (Sinhala):** අපේ app එක වැඩ කරන්න static sample data ටිකක් ඕනේ. ඒ සඳහා `topics.js` එකක් හදාගන්නවා.
- **Code Block:**

```
// src/data/topics.js
export const initialTopics = [
  {
    id: "jsx",
    title: "JSX",
    sprint: 2,
    done: false,
    summary: "JSX lets you write UI-like syntax inside JavaScript."
  },
  {
    id: "state",
    title: "State",
    sprint: 5,
    done: false,
    summary: "State is React memory for data that changes."
  },
  {
    id: "routing",
    title: "Routing",
    sprint: 9,
    done: false,
    summary: "Routing connects URLs to React components."
  }
];
```

- **Explanation of Fields (Sinhala):**
  - `id` : අනන්‍ය වූ අගයකි (unique string key). React keys වලට සහ dynamic routes වලට පාවිච්චි කරයි.
  - `title` : Topic එකේ නම.
  - `sprint` : ඒ topic එක ඉගෙනගත්ත sprint එක.
  - `done` : boolean අගයකි. Topic එක ඉවරද නැද්ද කියලා බලාගන්න පාවිච්චි කරයි.
  - `summary` : Topic එක ගැන කෙටි විස්තරයක්.

S10. Rendering Data with map() (map() භාවිතයෙන් Data render කිරීම)

- **Concept Explanation (Sinhala):** Array එකක තියෙන data ටික JSX බවට පත් කරලා screen එකේ පෙන්වන්න React වලදී අපි බහුලවම පාවිච්චි කරන්නේ `map()` function එකයි.
- **Code Block:**

```
{topics.map((topic) => (
  <article key={topic.id}>
    <h2>{topic.title}</h2>
    <p>Sprint {topic.sprint}</p>
    <p>{topic.done ? "Completed" : "Not completed"}</p>
  </article>
))}
```

- **Why key={topic.id} matters (Sinhala):** React වලට list එකක තියෙන elements වෙන වෙනම track කරගන්න අනිවාර්යයෙන්ම **stable key** එකක් ඕනේ.
  - හොඳම key එක: `key={topic.id}` (stable unique ID එකක්).
  - දුර්වල key එක: `key={index}` (list එකේ order එක වෙනස් උනොත් ප්‍රශ්න ඇතිවේ).
  - නරකම key එක: `key={Math.random()}` (හැම render එකකදීම අලුත් key එකක් හැදෙන නිසා performance සෑහෙන්න බහිනවා).

S11. Conditional Rendering (කොන්දේසි මත UI පෙන්වීම)

- **Concept Explanation (Sinhala):** ලැබෙන data හෝ state එක අනුව වෙනස් UI කොටස් පෙන්වීමට conditional rendering පාවිච්චි කරනවා.
- **Code Block:**

```
// 1. Ternary Operator ( ? : ) භාවිතය:
<p>{topic.done ? "Completed" : "Not completed"}</p>

// 2. Short-circuit Evaluation ( && ) භාවිතය:
{submittedPlan && (
  <section>
    <h2>Current plan</h2>
    <p>{submittedPlan}</p>
  </section>
)}
```

මෙතනදී `submittedPlan` එකක් තිබුණොත් විතරක් ඒ `section` එක render වෙනවා. නැත්නම් කිසිම දෙයක් පෙන්වන්නේ නැහැ.

S12. Routing Setup (React Router සැකසීම)

- **Concept Explanation (Sinhala):** පිටු අතර මාරු වීමට (navigation) React Router අපේ `App.jsx` එක ඇතුළත setup කරගන්නේ මෙහෙමයි.
- **Code Block:**

```
// src/App.jsx
import { BrowserRouter, Routes, Route } from "react-router";
import { LearningProvider } from "./context/LearningContext";
import Layout from "./components/Layout";
import Dashboard from "./pages/Dashboard";
import TopicList from "./pages/TopicList";
import TopicDetail from "./pages/TopicDetail";
import PracticePlanner from "./pages/PracticePlanner";
import NotFound from "./pages/NotFound";

export default function App() {
  return (
    <BrowserRouter>
      <LearningProvider>
        <Routes>
          <Route element={<Layout />}>
            <Route path="/" element={<Dashboard />} />
            <Route path="/topics" element={<TopicList />} />
            <Route path="/topics/:topicId" element={<TopicDetail />} />
            <Route path="/practice" element={<PracticePlanner />} />
            <Route path="*" element={<NotFound />} />
          </Route>
        </Routes>
      </LearningProvider>
    </BrowserRouter>
  );
}
```

S13. Layout Route (Layout Route සහ Outlet භාවිතය)

- **Concept Explanation (Sinhala):** හැම පිටුවකම එකම Navbar එකක් සහ එකම structure එකක් share කරගන්න Layout routes පාවිච්චි කරනවා. මෙතනදී `Outlet` එකෙන් කරන්නේ child routes වල content පෙන්විය යුතු තැන තීරණය කරන එකයි.
- **Code Block:**

```
// src/components/Layout.jsx
import { Outlet } from "react-router";
import Navbar from "./Navbar";

export default function Layout() {
  return (
    <>
      <Navbar />
      <main className="container">
        <Outlet />
      </main>
    </>
  );
}
```

S14. Navigation (පිටු අතර යාම)

- **Concept Explanation (Sinhala):** React Router වල navigation සඳහා `NavLink` හෝ `Link` පාවිච්චි කරනවා.
  - **රීතිය (Rule):** පිටු අතර මාරු වීමට (navigation) හැමවිටම links පාවිච්චි කරන්න. යම් action එකක් (උදා: data save කිරීමක්) කරන්න විතරක් buttons පාවිච්චි කරන්න.
- **Code Block:**

```
// src/components/Navbar.jsx
import { NavLink } from "react-router";

export default function Navbar() {
  return (
    <nav aria-label="Main navigation">
      <NavLink to="/">Dashboard</NavLink>
      <NavLink to="/topics">Topics</NavLink>
      <NavLink to="/practice">Practice</NavLink>
    </nav>
  );
}
```

S15. Dynamic Routes (Dynamic Routes යනු මොනවාද?)

- **Concept Explanation (Sinhala):** URL එකේ වෙනස් වන කොටසක් (variable/parameter) අඩංගු වන routes වලට dynamic routes කියනවා.
  - උදා: `<Route path="/topics/:topicId" element={<TopicDetail />} />`
  - මෙහි `:topicId` යනු ඕනෑම අගයක් විය හැක. (උදා: `/topics/jsx`, `/topics/state`). මෙවිට වෙන වෙනම routes දහස් ගණන් ලියන්න අවශ්‍ය වෙන්නේ නැහැ, එකම component template එකෙන් වැඩේ කරන්න පුළුවන්.

S16. Reading URL Parameters with useParams() (URL Parameters කියවීම)

- **Concept Explanation (Sinhala):** Dynamic route එකෙන් URL එකට එන අගය අපේ component එක ඇතුළතදී කියවා ගැනීමට `useParams()` කියන hook එක පාවිච්චි කරනවා.
- **Code Block:**

```
import { useParams } from "react-router";

export default function TopicDetail() {
  const { topicId } = useParams();
  return <h1>Topic ID: {topicId}</h1>;
}
```

වැදගත්: URL params හැමවිටම ලැබෙන්නේ **String** වර්ගයෙන් බව මතක තබාගන්න.

S17. Topic List Page (සියලුම Topics පෙන්වන පිටුව)

- **Concept Explanation (Sinhala):** Context එකෙන් `topics` array එක ලබාගෙන, එය `map()` කර, topics ලැයිස්තුවක් screen එකේ පෙන්වන ආකාරය මෙහි දැක්වේ.
- **Code Block:**

```
// src/pages/TopicList.jsx
import { Link } from "react-router";
import { useLearning } from "../context/LearningContext";

export default function TopicList() {
  const { topics } = useLearning();

  return (
    <main>
      <h1>Topics</h1>
      <section aria-label="React learning topics">
        {topics.map((topic) => (
          <article key={topic.id}>
            <h2>{topic.title}</h2>
            <p>Sprint {topic.sprint}</p>
            <p>{topic.done ? "Completed" : "Not completed"}</p>
            <Link to={`/topics/${topic.id}`}>View topic</Link>
          </article>
        ))}
      </section>
    </main>
  );
}
```

S18. Topic Detail Page (එක් එක් Topic එකේ විස්තර පිටුව)

- **Concept Explanation (Sinhala):** URL එකෙන් ලැබෙන `topicId` එක පාවිච්චි කරලා අදාළ topic එක array එකෙන් සොයාගෙන (`find()`), එහි විස්තර සහ එය complete කිරීමට අදාළ button එක පෙන්වන පිටුව මෙයයි.
- **Code Block:**

```
// src/pages/TopicDetail.jsx
import { useParams, Link } from "react-router";
import { useLearning } from "../context/LearningContext";

export default function TopicDetail() {
  const { topicId } = useParams();
  const { topics, markDone } = useLearning();

  const topic = topics.find((item) => item.id === topicId);

  if (!topic) {
    return (
      <main>
        <h1>Topic not found</h1>
        <Link to="/topics">Back to topics</Link>
      </main>
    );
  }

  return (
    <main>
      <h1>{topic.title}</h1>
      <p>Sprint {topic.sprint}</p>
      <p>{topic.summary}</p>
      <p>Status: {topic.done ? "Completed" : "Not completed"}</p>
      <button
        type="button"
        onClick={() => markDone(topic.id)}
        disabled={topic.done}
      >
        {topic.done ? "Already completed" : "Mark as completed"}
      </button>
    </main>
  );
}
```

S19. 404 Not Found Page (නොපවතින පිටු සඳහා පිටුවක්)

- **Concept Explanation (Sinhala):** User වැරදි URL එකක් type කළොත්, හිස් හෝ broken screen එකක් වෙනුවට 404 error page එකක් පෙන්වීම හොඳ developer පුරුද්දක්. Router එකේ `<Route path="*" element={<NotFound />} />` මගින් මෙය සිදුකරයි.
- **Code Block:**

```
// src/pages/NotFound.jsx
import { Link } from "react-router";

export default function NotFound() {
  return (
    <main>
      <h1>404</h1>
      <p>Page not found.</p>
      <Link to="/">Back to dashboard</Link>
    </main>
  );
}
```

S20. Shared State with Context (Context API මගින් දත්ත බෙදාගැනීම)

- **Concept Explanation (Sinhala):** අපේ app එකේ විවිධ පිටු වලට එකම data අවශ්‍ය වෙනවා (උදා: Dashboard එකට progress බලන්න topics ඕනේ, TopicList එකට topics පෙන්වන්න ඕනේ, Detail page එකට topic එකක් update කරන්න ඕනේ).

  Prop drilling (එක component එකක ඉඳන් තව එකකට prop පාස් කර කර යෑම) නැති කරන්න අපි **Context API** පාවිච්චි කරනවා. Context එකෙන් කෙලින්ම ඕනෑම component එකකට data ලබාදෙන්න පුළුවන්.

S21. Learning Context Setup (Context සහ Provider සැකසීම)

- **Concept Explanation (Sinhala):** `createContext()` සහ `useContext()` පාවිච්චි කරලා shared state එකක් සහ custom hook එකක් හදන හැටි පහත පරිදි වේ.
- **Code Block:**

```
// src/context/LearningContext.jsx
import { createContext, useContext, useState } from "react";
import { initialTopics } from "../data/topics";

const LearningContext = createContext(null);

export function LearningProvider({ children }) {
  const [topics, setTopics] = useState(initialTopics);

  function markDone(id) {
    setTopics((prevTopics) =>
      prevTopics.map((topic) =>
        topic.id === id ? { ...topic, done: true } : topic
      )
    );
  }

  return (
    <LearningContext.Provider value={{ topics, markDone }}>
      {children}
    </LearningContext.Provider>
  );
}

export function useLearning() {
  const value = useContext(LearningContext);
  if (!value) {
    throw new Error("useLearning must be used inside LearningProvider");
  }
  return value;
}
```

S22. Context Should Not Be Overused (Context ඕනවට වඩා පාවිච්චි කරන්න එපා)

- **Concept Explanation (Sinhala):** හැම කුඩා state එකක්ම Context එකට දාන්න එපා. Context පාවිච්චි කළ යුත්තේ app එක පුරාම share කරන්න ඕන ගෝලීය (global) දත්ත වලට විතරයි.
  - **Good Context Examples:** Logged-in user, theme mode, language settings, topic completion state.
  - **Bad Context Examples:** Form එකක් ඇතුළේ තියෙන text input value එකක්, button hover state එකක්.
  - **අපේ App එකේදී:** `topics` සහ `markDone` Context එකටත්, `practice planner text` සහ `form validation state` අදාළ component එකේ local state එකටත් දානවා.

S23. Immutable State Updates (නොවෙනස්වන සුළු State යාවත්කාලීන කිරීම)

- **Concept Explanation (Sinhala):** React වලදී අපි කවදාවත් පරණ array එකක් හෝ object එකක් කෙලින්ම modify (mutate) කරන්නේ නැහැ. හැමවිටම අලුත් copy එකක් හදලා තමයි state එක update කරන්නේ.
- **Code Block:**

```
// ❌ වැරදි ක්‍රමය (Direct Mutation):
function markDone(id) {
  const topic = topics.find((t) => t.id === id);
  topic.done = true; // පරණ object එක කෙලින්ම වෙනස් කළා
  setTopics(topics); // පරණ array reference එකම නිසා React re-render වෙන්නේ නැති වෙන්න පුළුවන්
}

// නිවැරදි ක්‍රමය (Immutable Update):
function markDone(id) {
  setTopics((prevTopics) =>
    prevTopics.map((topic) =>
      topic.id === id ? { ...topic, done: true } : topic
    )
  );
}
```

මෙහිදී `map()` සහ object spread operator (`...`) පාවිච්චි කරලා අලුත්ම array එකක් සහ අලුත්ම object එකක් සාදා React එකට ලබා දේ.

S24. Dashboard Page (Dashboard පිටුව)

- **Concept Explanation (Sinhala):** Dashboard එක මගින් මුළු topics ප්‍රමාණයෙන් කොපමණ ප්‍රමාණයක් ඉවරද කියලා summary එකක් බලාගන්න පුළුවන්.
- **Code Block:**

```
// src/pages/Dashboard.jsx
import { useLearning } from "../context/LearningContext";
import { countDone } from "../utils/countDone";

export default function Dashboard() {
  const { topics } = useLearning();
  const completed = countDone(topics);
  const total = topics.length;

  return (
    <main>
      <h1>React Learning Dashboard</h1>
      <p>
        You completed {completed} of {total} topics.
      </p>
      <progress value={completed} max={total}>
        {completed} / {total}
      </progress>
    </main>
  );
}
```

S25. Derived Values (derived values යනු මොනවාද?)

- **Concept Explanation (Sinhala):** දැනට තියෙන state එකකින් calculate කරලා ගන්න පුළුවන් අගයන් වලට **Derived Values** කියනවා.
  - උදාහරණයක් විදිහට `completed` count එක තියාගන්න වෙනම `[completed, setCompleted] = useState(0)` කියලා state එකක් අවශ්‍ය නැහැ. මොකද අපිට `topics` state එකෙන් ඒක කෙලින්ම calculate කරගන්න පුළුවන්.
  - **රීතිය:** State දෙකක් එකිනෙක sync කර කර ඉන්න යන්න එපා. එක state එකකින් අනිත් අගය calculate කරලා ගන්න (derived state).

S26. Utility Function: countDone() (Utility Functions)

- **Concept Explanation (Sinhala):** React වලින් තොරව වැඩ කරන සරල pure helper functions අපිට `utils/` folder එකේ ලියන්න පුළුවන්. මේවා ලේසියෙන්ම unit test කරන්න පුළුවන්.
- **Code Block:**

```
// src/utils/countDone.js
export function countDone(items) {
  return items.filter((item) => item.done).length;
}
```

S27. Pure Functions (Pure Functions යනු මොනවාද?)

- **Concept Explanation (Sinhala):** **Pure Function** එකක් කියන්නේ:
  - දෙන input එකට හැමවිටම එකම output එක දෙන,
  - වෙනත් බාහිර දත්ත modify නොකරන (side-effects නැති) function එකකටයි.
  - උදා: `countDone([{ done: true }, { done: false }])` හැම තිස්සෙම `1` කියන output එක දෙනවා. මේවා තමයි unit testing වලදී ප්‍රධානම target වෙන්නේ.

S28. Practice Planner Form (Practice Planner පිටුව සහ Form එක)

- **Concept Explanation (Sinhala):** User ට දිනපතා practice කරන්න ඕන දේවල් ලියා තැබීමට controlled input එකක් සහිත form එකක් මෙසේ නිර්මාණය කරයි.
- **Code Block:**

```
// src/pages/PracticePlanner.jsx
import { useState } from "react";

export default function PracticePlanner() {
  const [plan, setPlan] = useState("");
  const [submittedPlan, setSubmittedPlan] = useState("");

  function handleSubmit(event) {
    event.preventDefault();

    if (!plan.trim()) {
      return;
    }

    setSubmittedPlan(plan);
    setPlan("");
  }

  return (
    <main>
      <h1>Practice Planner</h1>
      <form onSubmit={handleSubmit}>
        <label htmlFor="plan">What will you practice today?</label>
        <textarea
          id="plan"
          value={plan}
          onChange={(event) => setPlan(event.target.value)}
        />
        <p>Characters: {plan.length}</p>
        <button type="submit" disabled={!plan.trim()}>
          Save practice plan
        </button>
      </form>

      {submittedPlan && (
        <section>
          <h2>Current plan</h2>
          <p>{submittedPlan}</p>
        </section>
      )}
    </main>
  );
}
```

S29. Controlled Inputs (Controlled Inputs ක්‍රියාවලිය)

- **Concept Explanation (Sinhala):** **Controlled Input** එකක් කියන්නේ input එකේ තියෙන value එක සම්පූර්ණයෙන්ම React State එකක් මගින් පාලනය වෙන එකටයි.
  - **ක්‍රියාවලිය (Cycle):** `User types` ➔ `onChange event` එක run වේ ➔ `setPlan()` මගින් `state` එක `update` වේ ➔ `React re-render` වේ ➔ `textarea` එක අලුත් `state value` එක පෙන්වයි.
  - **ප්‍රයෝජන:** Live character count පෙන්වන්න, validation කරන්න, සහ submit කළාට පස්සේ form එක clear කරන්න ලේසියි.

S30. Form Submission (Form Submit කිරීමේදී preventDefault())

- **Concept Explanation (Sinhala):** React SPA (Single Page Application) එකකදී form එකක් submit වෙද්දී මුළු page එකම reload වෙන එක නතර කරන්න `event.preventDefault()` අනිවාර්යයෙන්ම පාවිච්චි කළ යුතුය. නැතහොත් React App එකේ දැනට තියෙන state සියල්ල නැති වී යයි.

S31. Form Validation (Form එකේ වැරදි පරීක්ෂා කිරීම)

- **Concept Explanation (Sinhala):** හිස් Space විතරක් (empty inputs) submit කිරීම වැළැක්වීමට `.trim()` පාවිච්චි කරයි.
  - `"  React  ".trim()` ➔ `"React"` බවට පත්වේ.
  - empty spaces පමණක් ඇතුළත් කළ විට button එක disable කර තැබීමට `<button type="submit" disabled={!plan.trim()}>` පාවිච්චි කරයි.

S32. Local State vs Shared State (State තැබිය යුතු නිවැරදි තැන තීරණය කිරීම)

- **Concept Explanation (Sinhala):**
  - **Local State:** දත්ත අවශ්‍ය වන්නේ එකම එක component එකකට පමණක් නම් එය local state එකක් ලෙස තබන්න (උදා: `plan` input text එක).
  - **Shared State:** දත්ත සහෝදර (sibling) components කිහිපයකට හෝ app එකේ විවිධ තැන්වලට අවශ්‍ය නම් එය "Lift state up" කරන්න හෝ Context API එකකට දාන්න (උදා: `topics` array එක).

S33. Custom Hook Requirement (Custom Hook එකක් යනු කුමක්ද?)

- **Concept Explanation (Sinhala):** නැවත නැවත පාවිච්චි කරන්න පුළුවන් hook logic එකක් වෙනම function එකකට ලියලා, ඒකේ නම `use` කියන වචනයෙන් ආරම්භ කිරීමට **Custom Hooks** කියනවා (උදා: `useLearning`, `useFetch`).

S34. Basic useFetch() Hook (Fetch කිරීම සඳහා Custom Hook එකක්)

- **Concept Explanation (Sinhala):** API එකකින් data fetch කරගන්නා ආකාරය සහ එහි loading, error සහ data states වෙන වෙනම handle කරන හැටි මෙසේ ලියන්න පුළුවන්.
- **Code Block:**

```
// src/hooks/useFetch.js
import { useEffect, useState } from "react";

export function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    async function loadData() {
      try {
        setLoading(true);
        setError(null);
        const response = await fetch(url);
        if (!response.ok) {
          throw new Error("Request failed");
        }
        const json = await response.json();
        setData(json);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    }
    loadData();
  }, [url]);

  return { data, loading, error };
}
```

S35. Loading, Error, and Data UI (පරිශීලකයාට Feedback ලබාදීම)

- **Concept Explanation (Sinhala):** Data fetch වෙනකන් screen එක බිලන්ක් (blank) එකේ තියන්නේ නැතුව පැහැදිලි UI feedback එකක් දීම අත්‍යවශ්‍යයි.
- **Code Block:**

```
if (loading) {
  return <p>Loading...</p>;
}

if (error) {
  return <p role="alert">Error: {error}</p>;
}

return <div>{/* Render your data here */}</div>;
```

S36. Beginner Backend Integration Idea (Backend එකක් සමඟ වැඩ කරන හැටි)

- **Concept Explanation (Sinhala):** Sprint 16 වලදී full backend එකක් හැදීම අවශ්‍ය නැහැ, frontend එකෙන් backend API එකකට data යවන flow එක මෙහෙමයි කියලා දැනගෙන ඉන්න ඕනේ: `React Component` ➔ `Custom Hook/Service file` ➔ `Fetch Request` ➔ `Backend API` ➔ `JSON Response` ➔ `React State Updates` ➔ `UI Re-renders`.

  API requests හැමවිටම වෙනම Service file එකක ලියා තැබීම හොඳ පුරුද්දක්.
- **Code Block:**

```
// src/services/api.js
const API_BASE_URL = import.meta.env.VITE_API_BASE_URL;

export async function getTopics() {
  const response = await fetch(`${API_BASE_URL}/topics`);
  if (!response.ok) {
    throw new Error("Failed to load topics");
  }
  return response.json();
}
```

S37. Environment Variables (පරිසර විචල්‍යයන්)

- **Concept Explanation (Sinhala):** API URL වගේ configurations source code එක ඇතුළේ hardcode නොකර බාහිරව තබා ගැනීමට `.env` files පාවිච්චි කරයි.
  - Vite app එකකදී මෙම විචල්‍යයන් `VITE_` යන prefix එකෙන් ආරම්භ විය යුතුය.
  - උදා: `VITE_API_BASE_URL=https://api.example.com`
  - Code එක ඇතුළතදී භාවිතය: `const API_BASE_URL = import.meta.env.VITE_API_BASE_URL;`
  - වැදගත් ආරක්ෂක උපදෙසක්: රහසිගත API keys කිසිවිටක frontend env variables වල දාන්න එපා (මොකද browser එකෙන් ඕනෑම කෙනෙක්ට මේවා බලාගන්න පුළුවන්).

S38. Beginner TypeScript Layer (TypeScript මූලික පියවර)

- **Concept Explanation (Sinhala):** Sprint 16 වලදී මුළු app එකම TypeScript කරන්න ඕනේ නැහැ, සරලව component එකකට props pass කරද්දී ඒවායේ types ආරක්ෂා කරගන්න (type safety) TypeScript පාවිච්චි කරන්න ඉගෙනගමු.
- **Code Block:**

```
// src/components/Button.tsx
type ButtonProps = {
  label: string;
  onClick: () => void;
};

export default function Button({ label, onClick }: ButtonProps) {
  return (
    <button type="button" onClick={onClick}>
      {label}
    </button>
  );
}
```

S39. .tsx Files (.tsx සහ .ts වල වෙනස)

- **Concept Explanation (Sinhala):**
  - **.tsx:** File එක ඇතුළත TypeScript සමඟ JSX (HTML-like syntax) ලියවෙනවා නම් පාවිච්චි කරයි (උදා: `Button.tsx`).
  - **.ts:** File එක ඇතුළත TypeScript පමණක් ඇති, JSX නැති අවස්ථා වලදී පාවිච්චි කරයි (උදා: `countDone.ts`).
  - **.jsx:** සාමාන්‍ය JavaScript සහ JSX ඇති විට (TypeScript නැතිව).

S40. Typed Topic Card Example (TypeScript Props භාවිතයෙන් Component එකක්)

- **Concept Explanation (Sinhala):** Reusable component එකක් වන `TopicCard` එකට එන props වල types, TypeScript මගින් define කරන ආකාරය මෙසේය.
- **Code Block:**

```
// src/components/TopicCard.tsx
type TopicCardProps = {
  title: string;
  sprint: number;
  done: boolean;
};

export default function TopicCard({ title, sprint, done }:
TopicCardProps) {
  return (
    <article>
      <h2>{title}</h2>
      <p>Sprint {sprint}</p>
      <p>{done ? "Completed" : "Not completed"}</p>
    </article>
  );
}
```

දැන් ඔයා වැරදීමකින්වත් `title` එකට string එකක් වෙනුවට number එකක් පාස් කළොත් TypeScript මගින් error එකක් පෙන්වනු ඇත.

S41. Testing Requirement (Unit Test එකක් ලියන හැටි)

- **Concept Explanation (Sinhala):** අපේ `countDone()` utility function එක නිවැරදිව ක්‍රියාත්මක වෙනවාදැයි බැලීමට Vitest මගින් ලියන සරල unit test එකක් පහත පරිදි වේ.
- **Code Block:**

```
// src/utils/countDone.test.js
import { describe, expect, test } from "vitest";
import { countDone } from "./countDone";

describe("countDone", () => {
  test("counts completed topics", () => {
    const topics = [
      { id: "jsx", done: true },
      { id: "state", done: false },
      { id: "routing", done: true }
    ];
    expect(countDone(topics)).toBe(2);
  });

  test("returns 0 when no topics are completed", () => {
    const topics = [
      { id: "jsx", done: false },
      { id: "state", done: false }
    ];
    expect(countDone(topics)).toBe(0);
  });
});
```

S42. Test Syntax Review (ටෙස්ට් එකක මූලික හැඩය)

- **Concept Explanation (Sinhala):**
  - `describe()` : එක සමාන ටෙස්ට් කාණ්ඩ එකතු කර තබයි (Grouping).
  - `test()` : තනි ටෙස්ට් කේස් එකක් සාදයි.
  - `expect().toBe()` : අපට ලැබුණු සැබෑ අගය (actual value) සහ අප බලාපොරොත්තු වන අගය (expected value) සමානදැයි පරීක්ෂා කරයි.
  - හොඳ පුරුද්ද: ටෙස්ට් එකේ නම (description) කියවපු ගමන් ඒකෙන් වෙන්නේ මොකක්ද කියලා පැහැදිලි වෙන්න ඕනේ. (උදා: `test("counts completed topics")`).

S43. React Testing Library Strategy (RTL පරීක්ෂණ සැලසුම)

- **Concept Explanation (Sinhala):** React Testing Library (RTL) මගින් අපි හැමවිටම පරීක්ෂා කරන්නේ code එක ඇතුළේ වෙන දේවල් නෙවෙයි, පරිශීලකයාට (user) screen එකේ පේන සහ කරන්න පුළුවන් දේවල්ය.
  - උදාහරණයක් විදිහට: `PracticePlanner` render කරන්න ➔ textarea එකට text එකක් type කරන්න ➔ submit button එක click කරන්න ➔ අලුත්ම පිටු plan එක screen එකේ පේන්න තියෙනවාද කියලා check කරන්න.

S44. Sprint 16 Test Strategy (ටෙස්ට් කිරීමේ උපාය මාර්ගය)

- **Concept Explanation (Sinhala):** සම්පූර්ණ app එකක් ටෙස්ට් කරද්දී අපිට පැහැදිලි සැලැස්මක් (Strategy) තිබිය යුතුය.
  - **Unit test:** සරල pure functions (`countDone` වැනි) ටෙස්ට් කිරීමට.
  - **Functional test:** Component එකක් ඇතුළත ක්‍රියාවන් (Form submit, list rendering) නිවැරදිද බැලීමට.
  - **E2E test:** සම්පූර්ණ පරිශීලක ගමනක් (User journey) ආරම්භයේ සිට අවසානය දක්වා (Topics පිටුවට ගොස්, එකක් complete කර, Dashboard එකේ progress වෙනස් වුවාදැයි බැලීම) පරීක්ෂා කිරීමට.
  - **Accessibility check:** Heading නිවැරදිද, labels තියෙනවද, keyboard එකෙන් වැඩ කරන්න පුළුවන්ද කියා බැලීමට.
  - **Performance check:** Lighthouse හෝ DevTools මගින් app එකේ වේගය බැලීමට.

S45. E2E Test Thinking (End-to-End පරීක්ෂණ සිතීම)

- **Concept Explanation (Sinhala):** E2E පරීක්ෂණයකින් මගින් මුළු පද්ධතියම එකට සම්බන්ධ වී ක්‍රියාකරන ආකාරය බලයි (routing, links, dynamic routes, context updates, dashboard values).
  - කුඩා pure functions (උදා: `countDone()`) E2E වලින් ටෙස්ට් කිරීමට යාම නාස්තියකි. ඒවා සරල unit tests වලින් නිම කළ යුතුය.

S46. Regression Testing Thinking (Regression ටෙස්ට් යනු මොනවාද?)

- **Concept Explanation (Sinhala):** **Regression** එකක් කියන්නේ කලින් හොඳට වැඩ කරපු දෙයක්, අලුතින් කේතය වෙනස් කළාට පස්සේ (Refactoring) වැරදීමකින් බිඳී යාමයි.
  - Regression tests මගින් අප අලුතින් code වෙනස් කරද්දී පරණ හැඩ කැඩී තිබේදැයි ස්වයංක්‍රීයව පරීක්ෂා කර අනතුරු ආරක්ෂා කරයි.

S47. Accessibility Requirements (Accessibility - සැමට පොදු නිර්මාණය)

- **Concept Explanation (Sinhala):** App එකක් හදද්දී screen readers පාවිච්චි කරන අයට සහ keyboard එකෙන් පමණක් වැඩ කරන අයටත් පහසුවෙන් පාවිච්චි කළ හැකි විය යුතුය.
  - **Semantic HTML පාවිච්චි කරන්න:** `<div>` ගොඩවල් වෙනුවට `<main>`, `<section>`, `<article>`, `<nav>` වැනි අර්ථවත් tags පාවිච්චි කරන්න.
  - **Form inputs වලට labels අනිවාර්යයෙන්ම සම්බන්ධ කරන්න:** `<label htmlFor="plan">` සහ `<textarea id="plan">`. placeholder එකක් කියන්නේ label එකකට ආදේශකයක් නොවේ.
  - **Actions වලට real buttons පාවිච්චි කරන්න:** `<div>` එකකට onClick දානවා වෙනුවට `<button>` පාවිච්චි කරන්න.

S48. Keyboard Accessibility (Keyboard එකෙන් වැඩ කළ හැකිද?)

- **Concept Explanation (Sinhala):** Mouse එකක් නැතුව Tab key එකෙන් පමණක් links සහ buttons වෙත යා හැකිදැයි පරීක්ෂා කරන්න.
  - CSS වලදී `outline: none` දමා focus border එක සම්පූර්ණයෙන්ම නැති කිරීම ඉතා නරක පුරුද්දක්. ඒ වෙනුවට පැහැදිලි custom focus style එකක් දෙන්න:

```
button:focus-visible {
  outline: 2px solid currentColor;
  outline-offset: 2px;
}
```

S49. Performance Requirements (Performance - වේගය සහ ප්‍රතිචාරය)

- **Concept Explanation (Sinhala):**
  - **Real Performance (සැබෑ වේගය):** Load time එක, network requests ගණන, bundle size එක ආදිය මැනිය හැකි අගයන්ය.
  - **Perceived Performance (දැනෙන වේගය):** App එක ඇත්තටම වැඩ කරනවාට වඩා, user ට දැනෙන වේගයයි (උදා: data fetch වෙනකන් Loading spinner එකක් හෝ message එකක් පෙන්වීමෙන් user ට බලාගෙන ඉන්න පහසු වේ).

S50. Loading Feedback (පරිශීලකයා අතරමං නොකිරීම)

- **Concept Explanation (Sinhala):** යම් දෙයක් save වෙද්දී හෝ fetch වෙද්දී button එක disable කර "Saving..." හෝ "Loading..." ලෙස පෙන්වීමෙන් පරිශීලකයා නැවත නැවත button එක click කිරීම වළක්වා ගත හැක.

S51. Avoiding Layout Shift (Layout Shift වැළැක්වීම)

- **Concept Explanation (Sinhala):** Page එක load වෙද්දී පින්තූරයක් පසුව load වීම නිසා අකුරු එකපාරටම පහළට පැනීම layout shift (CLS - Cumulative Layout Shift) ලෙස හඳුන්වයි.
  - මෙය වැළැක්වීමට `<img>` tag එකට `width` සහ `height` කලින්ම ලබා දී ඉඩ වෙන් කරන්න.

S52. Lazy Loading Thinking (අවශ්‍ය විට පමණක් Load කිරීම)

- **Concept Explanation (Sinhala):** සම්පූර්ණ app එකම එක පාර load නොකර, අදාළ page එකට යද්දී පමණක් ඒ page එක load වීමට සැලැස්වීමට `lazy()` සහ `Suspense` පාවිච්චි කළ හැක.
- **Code Block:**

```
import { lazy, Suspense } from "react";
const PracticePlanner = lazy(() => import("./pages/PracticePlanner"));

// Usage inside Routes:
<Suspense fallback={<p>Loading page...</p>}>
  <PracticePlanner />
</Suspense>
```

වැදගත්: හැම පොඩි button එකක්ම lazy-load කරන්න යන්න එපා. එයින් app එක අනවශ්‍ය ලෙස සංකීර්ණ වේ.

S53. Styling Requirement (Styling වලදී අනුගමනය කළ යුතු දේ)

- **Concept Explanation (Sinhala):** Plain CSS, CSS Modules, SCSS, හෝ Tailwind CSS ඕනෑම එකක් පාවිච්චි කළ හැක. වැදගත්ම දේ වන්නේ සංගතිකභාවය (Consistency) යි.
  - හැම button එකක්ම එකම වගේ පෙනීම, margins/paddings සමාන මට්ටමක තබා ගැනීම මෙහිදී වැදගත් වේ.

S54. Component Reuse (Components නැවත භාවිතය)

- **Concept Explanation (Sinhala):** එකම UI කොටස නැවත නැවත ලියනවා වෙනුවට එය වෙනම component එකක් ලෙස හදා props හරහා වෙනස් දත්ත පාස් කරන්න.
  - උදාහරණයක් ලෙස: `TopicCard` එකක් හදාගෙන එය `TopicList` එකේ map එක ඇතුළත නැවත පාවිච්චි කිරීම.

S55. Props in Sprint 16 (Props භාවිතය)

- **Concept Explanation (Sinhala):** Props මගින් parent component එකක සිට child component එකක් වෙත දත්ත ලබා දේ.
- **Code Block:**

```
// Parent: TopicList.jsx
<TopicCard topic={topic} />

// Child: TopicCard.jsx
export default function TopicCard({ topic }) {
  return <h2>{topic.title}</h2>;
}
```

S56. Children in Sprint 16 (children prop එක භාවිතය)

- **Concept Explanation (Sinhala):** Component එකක් ඇතුළත වෙනත් JSX elements වට කර තැබීමට (wrap) `children` prop එක පාවිච්චි කරයි.
- **Code Block:**

```
function PageSection({ title, children }) {
  return (
    <section>
      <h2>{title}</h2>
      {children}
    </section>
  );
}

// Usage:
<PageSection title="Current plan">
  <p>Practice routing and Context today.</p>
</PageSection>
```

S57. Events in Sprint 16 (React Events handle කිරීමේ රීති)

- **Concept Explanation (Sinhala):** React වලදී events ලියද්දී:
  - camelCase භාවිත කරන්න (`onClick` , `onChange`).
  - Argument එකක් පාස් කිරීමට අවශ්‍ය නම් එය arrow function එකක් ඇතුළත ලියන්න.
- **Code Block:**

```
// ❌ වැරදියි (render වෙද්දීම function එක run වේ):
onClick={markDone(topic.id)}

// නිවැරදියි (click කළ විට පමණක් run වේ):
onClick={() => markDone(topic.id)}
```

S58. Effects in Sprint 16 (useEffect භාවිතය)

- **Concept Explanation (Sinhala):** බාහිර දත්ත ලබා ගැනීම (Data fetching), document title එක වෙනස් කිරීම වැනි side-effects සඳහා `useEffect` පාවිච්චි කරයි.
  - **රීතිය:** `useEffect` එකේ direct callback එක `async` කරන්න එපා. `async` function එක ඇතුළත ලියා එය එහිදීම call කරන්න.
- **Code Block:**

```
// ❌ වැරදියි (Do not make the effect callback directly async):
useEffect(async () => {
  const response = await fetch(url);
}, [url]);

// නිවැරදියි:
useEffect(() => {
  async function loadData() {
    const response = await fetch(url);
  }
  loadData();
}, [url]);
```

S59. Dependency Arrays (useEffect Dependency Arrays)

- **Concept Explanation (Sinhala):** Dependency array එකෙන් කියන්නේ මේ effect එක නැවත run වෙන්න ඕනේ මොන වගේ වෙනස්වීම් මතද කියන එකයි.
  - හිස් array එකක් `[]` දුන්නොත් එය run වෙන්නේ component එක මුලින්ම load වෙද්දී (first mount) පමණි.
  - යම් විචල්‍යයක් (variable) array එකට ඇතුළත් කළහොත් (උදා: `[url]`), ඒ අගය වෙනස් වන හැම වතාවකදීම effect එක නැවත ක්‍රියාත්මක වේ.

S60. Memoization (අනවශ්‍ය Performance Optimization වලින් වැළකීම)

- **Concept Explanation (Sinhala):** කුඩා app එකක් සඳහා හැමතැනටම `useMemo` හෝ `useCallback` දාන්න යන්න එපා. ඒවා පාවිච්චි කළ යුත්තේ සැබැවින්ම බරපතල ගණනය කිරීම් (expensive calculations) තිබේ නම් පමණි.
  - සරල `const completed = countDone(topics);` යන්න කුඩා app එකකට හොඳටම ප්‍රමාණවත් වේ.

S61. Debug Task: Fix Mutation (Mutation දෝෂය හඳුනාගෙන නිවැරදි කිරීම)

- **Concept Explanation (Sinhala):** පහත දැක්වෙන්නේ වැරදි සහ නිවැරදි state update කිරීම් ය. විභාග වලදී සහ code reviews වලදී මෙය නිතරම අසන ප්‍රශ්නයකි.
- **Code Block:**

```
// ❌ බිඳුණු/වැරදි කේතය (Broken Code):
function markDone(id) {
  const topic = topics.find((t) => t.id === id);
  topic.done = true;
  setTopics(topics);
}

// නිවැරදි කරන ලද කේතය (Fixed Code):
function markDone(id) {
  setTopics((prevTopics) =>
    prevTopics.map((topic) =>
      topic.id === id ? { ...topic, done: true } : topic
    )
  );
}
```

S62. Debug Task: Fix Route Import Problems (Routing වැරදි නිවැරදි කිරීම)

- **Concept Explanation (Sinhala):**
  - වැරැද්ද 1: Router එකේ path එකට element එකක් දීමේදී JSX tag එකක් ලෙස දිය යුතුය.
    - ❌ වැරදියි: `element={TopicDetail}`
    - නිවැරදියි: `element={<TopicDetail />}`
  - වැරැද්ද 2: React Router version එක අනුව imports වෙනස් විය හැක.
    - `import { BrowserRouter, Routes, Route } from "react-router";`

S63. Debug Task: Fix Missing Provider (නිතරම එන Context Error එකක්)

- **Concept Explanation (Sinhala):** `useLearning must be used inside LearningProvider` කියන error එක එන්නේ ඔයා Context එකක් පාවිච්චි කරන component එකක්, ඒ Context Provider එකෙන් වට (wrap) කරලා නැති වුණාමයි.
  - හැමවිටම Provider එක `App.jsx` වල හෝ ඉහළම මට්ටමක ඇති බව තහවුරු කරගන්න.

S64. Debug Task: Fix Form Button Type (Form එකේ Button වර්ග)

- **Concept Explanation (Sinhala):** Form එකක් ඇතුළේ තියෙන button එකක් click කළාම වැරදීමකින් form එක submit වීම වැළැක්වීමට හැමවිටම button type එක පැහැදිලිව ලියන්න.
  - Submit කිරීමට: `<button type="submit">Save</button>`
  - වෙනත් ක්‍රියාවලට: `<button type="button">Cancel</button>`

S65. Debug Task: Fix Missing Labels (Labels නොමැති වීමේ දෝෂය)

- **Concept Explanation (Sinhala):** placeholder එකක් විතරක් තිබීමෙන් screen readers වලට හඳුනාගන්න අමාරුයි. හැමවිටම input එකට label එකක් `htmlFor` හරහා සම්බන්ධ කරන්න.
- **Code Block:**

```
// ❌ දුර්වලයි:
<textarea placeholder="Practice plan" />

// විශිෂ්ටයි:
<label htmlFor="plan">What will you practice today?</label>
<textarea id="plan" />
```